AgentGuard Architectural Framework

BRING YOUR OWN AI AGENT (BYOA) GOVERNANCE GATEWAY

Category: iQOO Hackathon 2026 (Productivity)
Model: Thin Vertical Slice / Enforcement Gateway
Status: Technical Blueprint

1. CORE PRODUCT VISION & SYSTEM ARCHITECTURE

AgentGuard provides centralized access governance, zero-trust authorization, and non-repudiable audit trails for autonomous AI agents. Instead of building agents inside the platform, organizations connect existing agents (OpenAI SDK, CrewAI, LangGraph, or custom scripts) to AgentGuard's gateway to govern tool calls without compromising reasoning logic.

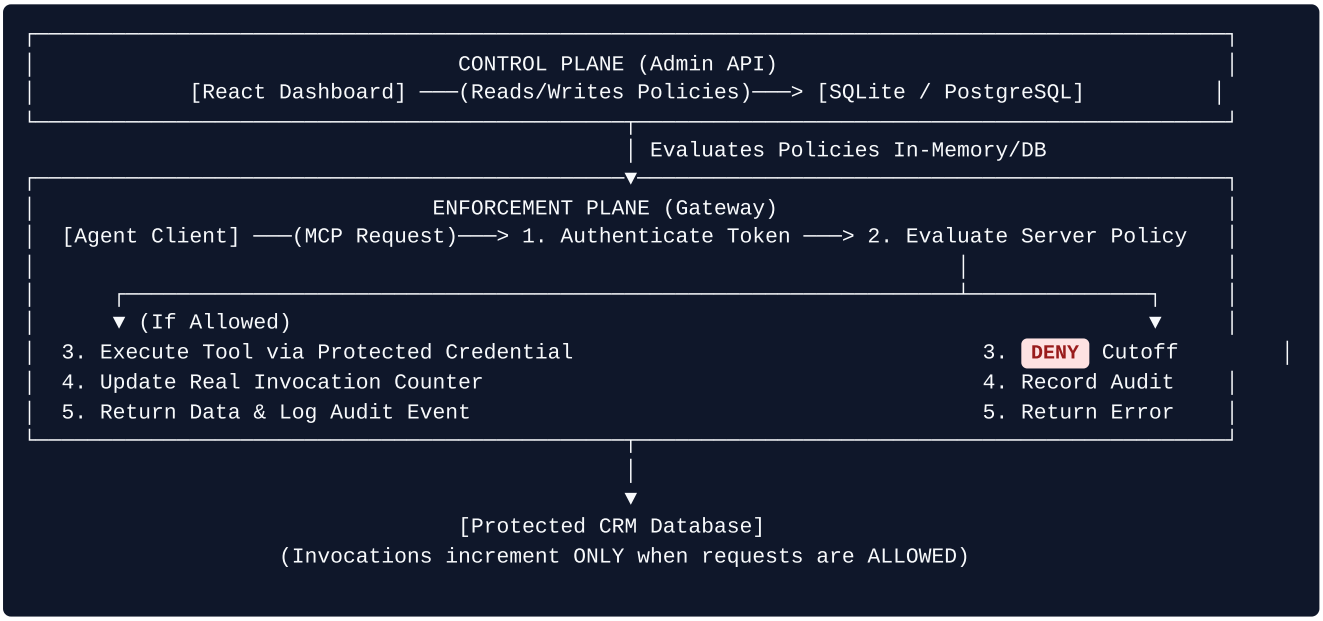
Control Plane (Administrative)

Manages agent identity registration, tool RBAC/ABAC policies, dynamic rule adjustment, human approval queues, and persistent audit trail visibility.

Enforcement Plane (Runtime Engine)

Intercepts MCP/API tool requests, authenticates tokens, executes server-side policy evaluation, enforces allow/deny gates, and records execution metrics.

2. END-TO-END ENFORCEMENT FLOW



3. THE 6-STAGE SECURITY PIPELINE

Stage	Technical Responsibility	Enforcement Mechanism
1. Identify	Agent Authentication	Validates incoming Bearer tokens against registered agent credentials; fails closed on unknown keys.
2. Authorize	Role & Policy Matching	Checks role-based permissions (`role` + `tool_name`) against server-managed policy rules.
3. Analyze	Deterministic Constraints	Evaluates parameters (e.g., query limits, bulk export tags) using predictable server-side rules.
4. Decide	Policy Engine Gate	Assigns explicit decision: `ALLOW`, `REQUIRE_APPROVAL`, or `DENY`.
5. Execute	Controlled Execution	Invokes protected resource using backend credentials ONLY if decision is `ALLOW`.

Stage	Technical Responsibility	Enforcement Mechanism
6. Audit	Non-Repudiable Logging	Records agent ID, tool, arguments, decision, execution result, and database counter state.

4. MINIMAL DATABASE SCHEMA (SQLITE / POSTGRESQL)

Control Plane Tables

```
-- Agents Table
CREATE TABLE agents (
    id TEXT PRIMARY KEY,
    name TEXT NOT NULL,
    token TEXT NOT NULL,
    role TEXT NOT NULL
);

-- Policies Table (Server-side Truth)
CREATE TABLE policies (
    role TEXT NOT NULL,
    tool_name TEXT NOT NULL,
    action TEXT NOT NULL, -- ALLOW,
REQUIRE_APPROVAL, DENY
    PRIMARY KEY (role, tool_name)
);
```

Audit & Resource Tables

```
-- Audit Logs Table
CREATE TABLE audit_logs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
    agent_id TEXT, tool_name TEXT,
    arguments TEXT, decision TEXT,
    execution_success BOOLEAN,
    counter_at_execution INTEGER
);

-- Protected Resource Invocation Counter
CREATE TABLE query_counter (
    id INTEGER PRIMARY KEY,
    total_invocations INTEGER
);
```

5. CORE SECURITY NON-NEGOTIABLES

1. **Credential Isolation:** Agents must never receive direct database connection strings or master tokens.
2. **Zero LLM Authorization:** Prompts or models cannot alter security policies or approve their own calls.
3. **Fail-Closed Design:** Unavailable policy entries or authorization failures default strictly to `DENY`.
4. **Uncompromising Proof:** Denied tool calls must never trigger backend database execution or increment counters.